BLUNDERMIND

Bot Controls — Technical Project Brief

Move Selection Mechanics, Formulas & Architecture

All controls described are available in the bot-control-panel.html sidebar. Formulas reference src/50-bot-engine.js and src/60-bot-ui.js. This brief reflects the state of the dev branch as of July 2026.

Section	Topic
1	Architecture Overview — Move Selection Pipeline
2	Move Source Layer (Opening Book / Maia3 / Stockfish / LCSF)
3	Think Time Calculation
4	Time Pressure Play Degradation (Independent Curves A & B)
5	Temperature & Move Sampling
6	Strategic Attractors (Centipawmeter)
7	Piece-Type Attractors
8	Move Distribution Filter & Luck
9	CP-Budget Acceptance, Degradation Guard & Hard Floor (Engine-Calculated)
10	Draw & Desperation Behaviours
11	Preset Personalities
12	Time Controls
13	Human Behaviour Flags
14	Appendix — Play From Any Position (Replay / Loaded Games / Invites)

1. Architecture Overview — Move Selection Pipeline

Every bot move passes through the sequential stages below. Each stage can veto, modify, or short-circuit to skip later stages. The stages run in strict order so that think time — computed in Stage 2 — feeds every downstream degradation calculation, and so the final engine-calculation stages (8–10) see the move the personality actually chose.

Stage	Name	What happens
1	Opening Book	ECO table lookup (preferred) or Lichess Masters API (mainline). If a matching move is found, it is played immediately after a brief book delay (400–1200 ms). Skipped once the game leaves known lines.
2	Think Time	Actual think time for this move computed from timing mode + Hustle attractor + clock pressure. Result fed as thinkSec into all downstream degradation curves.
3	Engine Query	Rough thinkSec estimate used to pick Maia ELO (curve A, if the ELO-degradation toggle is on). Maia3 or LCSF queried for candidate move probabilities.
4	Distribution Filter	Absolute min-probability floor removes near-zero candidates. Curve B (if the distribution toggle is on) narrows the upper percentile; the Luck attractor shifts the window (§8).
5	Attractor Reweighting	Each candidate move's probability is multiplied by exp(logBoost) where logBoost is the sum of all active strategic attractor signals. Sets an "applied" flag read by Stage 8.
6	Desperation (opt.)	If "seek stalemate when losing" is armed and the engine eval confirms a lost position, candidate weights are re-shaped toward low-own-mobility and material-dumping moves (§10).
7	Conviction Pick	The Conviction share of moves (default 0%): the top-ranked move of the reweighted distribution is played outright (argmax). The rest: final temperature applied (base T × phase/pressure modifiers) and one move is sampled (§5).
8	CP-Budget Acceptance	Personality bots only: if the pick differs from Maia's most-popular move, a shallow Stockfish probe checks that the pick loses ≤ CP budget vs the popular move; otherwise it walks down the preference order (§9a).
9	Degradation Guard	Switch, off by default (on for Bad Day): a sampled pick that evaluates better than Maia's top move is swapped back — popularity is not quality (§9b).
10	Hard Floor Backstop	Whatever mechanism produced the final pick, it may not lose more than the Hard Floor vs the popular move (unless the Floor is Off). Budget-cleared picks skip the check (§9c).

Key architectural principle #1 — think time drives degradation: Think time is computed first (Stage 2), and the resulting thinkSec value drives Stages 3–4 via the degradation curves. A Coffeehouse Hustler thinking 0.3 s on a move sees heavy ELO and distribution degradation on that specific move — not an average derived from the remaining clock.

Key architectural principle #2 — real centipawns, not probability: Maia probability measures how *popular* a move is at a rating, which is not the same as how *good* it is — at low ratings the two can even anticorrelate. Therefore the CP budget and all quality guarantees are enforced with actual Stockfish evaluations at Stages 8–10, never by treating a probability ratio as a centipawn value. Stages 8–9 run only for personality bots; the Hard Floor backstop covers every bot. All are fail-open (any probe timeout leaves the pick untouched).

2. Move Source Layer

2a. Opening Book

Two independent book modes are available. Either can be active simultaneously; each deactivates permanently the first time it fails to find a candidate move.

Mode	Source	Behaviour
Preferred	In-memory ECO table (~3 000 named openings)	Matches game history against ECO lines whose ECO code or family prefix is in the bot's slot list. Plays the next move from the highest-priority matching line. Exits the book at maxBookDepth (default 20 half-moves).
Mainline	Lichess Masters API (live)	Queries masters database for the most popular continuation from the current position. Falls back to Lichess database if no masters result. Plays the top-weighted move. Deactivates after the first cache miss or API error.

Book delay: 400–1200 ms random pause to simulate human reading time.

2b. Maia3 Neural Network

Maia3 is a series of neural networks (600–2600 ELO) trained to predict human moves at each rating level. The bot queries the appropriate ELO model (determined by degradation curve A) and receives a probability distribution over all legal moves.

- Model selection: effective ELO = pressureEffectiveMaiaEloByThink(thinkSec)
- ELO clamped to [600, 2600].
- Probability distribution filtered by the distribution range (§8), then reweighted by attractors (§6).

2c. Stockfish (SF) Engine

Used when the bot engine mode is set to Stockfish. Skill level 1–20 controlled by sfEffectiveLevel(). The engine is queried via WebWorker; MultiPV probes provide complexity scores used by the Chaos attractor.

```
sfEffectiveLevel: floor = pressureFloor
pressureFloor = max(1, startLevel − round(timePressureMaxDrop / 50))
timePressureMaxDrop = base ELO − curve A minimum (from the visible curve)
(or the sfPressureLevel slider when no max-drop is set)
```

The old blunder-limit-derived quality floor was removed — single-move quality is now guaranteed by the engine-calculated CP budget and the Hard Floor backstop (§9), not by a Stockfish skill floor.

Time degradation path (highest priority wins):

- Weaponizer active AND bot is ahead → return floor immediately
- Curve A present → spline interpolation in log-time space → lerp level to floor
- Fallback: linear ramp from startLevel at 30 s → floor at 0 s

2d. Level-Controlled Stockfish (LCSF)

LCSF uses Stockfish with controlled skill variation rather than Maia probability distributions. sfPickLevel() distributes calls around the target level:

```
p(offset) distribution:
−2: var2/2 (e.g. 10% at ±2 → 5% chance)
−1: var1/2 (e.g. 30% at ±1 → 15% chance)
0: 1 − var1 − var2 (remaining probability)
+1: var1/2
+2: var2/2
Level clamped to [1, 20]
```

var1 (±1 spread): 0–50%. **var2** (±2 spread): 0–20%. Setting both to 0 uses the exact target level every move.

3. Think Time Calculation

Think time is calculated in botThinkTime(moveProbs, clockMs) before the engine is queried. The result (thinkSec) drives all downstream degradation curves, so attractors that shorten think time (Hustler) directly increase move-quality degradation for that specific move.

3a. Timing Modes

Mode	Formula / Behaviour
Instant	Returns 0 ms. No delay whatsoever.
Fixed	thinkMs = botFixedDelayMs. Capped at min(fixed, clockMs × 0.08) when clock < 30 s. Capped at clockMs – 3 s when flagging is disabled.
Pace (default)	baseSec = (5 × 60) / pace. complexity = min(1 + entropy × 0.35, 2.5). thinkMs = baseSec × complexity × 1000. Pace slider: 5–120 (moves per 5 min equivalent).
Complexity	cplx = sfCplxScore if available, else min(1, entropy / 4). mult = botCplxMin + cplx × (botCplxMax – botCplxMin). thinkMs = botCplxBase × mult × 1000. Explicit base / min-mult / max-mult sliders in panel.
Mirror	avg = rolling average of human move timestamps. thinkMs = avg × jitter(0.8–1.2) × (1 + mirrorOffsetPct/100). Falls through to Complexity/Pace when no human moves recorded yet.

3b. Think Time Modifiers (applied in order)

Modifier	Condition	Effect
Weaponizer shortcut	Enabled AND bot clock > opp clock + leadMs	Return 0 ms immediately (skip all delays)
Move blink	botBehavBlink AND entropy < 0.5 (forced/obvious positions)	200–500 ms random; skips further calculation
Clock-pressure mirroring	botBehavClockMirror AND oppClock < botClock × 0.6	thinkMs ×= 0.5
Hustle attractor	value v ∈ [–5, +5]	thinkMs ×= (1 – v × 0.15). At +5: ×0.25 (very fast). At –5: ×1.75 (very slow).
Reconsideration	botBehavReconsider, 15% random chance	thinkMs ×= 1.5 to 2.5 (hesitation pause)
Base jitter	Always applied	thinkMs ×= uniform(0.8, 1.2)
Clock cap (low clock)	clockMs < 30 000 ms	thinkMs = min(thinkMs, clockMs × 0.08)
Flagging guard	botCanFlag = false	thinkMs = min(thinkMs, max(200, clockMs – 3000))
Global cap	Always applied	thinkMs = max(200, min(botThinkCapMs(), thinkMs)). Cap = max(6 s, min(45 s, startClock × 0.02)) — scales with time control.

Position entropy is Shannon entropy over Maia move probabilities: H = –Σ p·log■(p). Higher entropy = more complex / branching position.

4. Time Pressure Play Degradation

Two independent user-editable spline curves control how the bot's play quality degrades as think time per move decreases. Curve A's control points are seeded on a closed-form time-rating model (below); curve B's on a sigmoid. Both use log-scale interpolation on the X axis (think time in seconds) so that the visually equal spacing on the panel matches the mathematical interpolation.

Each curve has its own ON/OFF toggle beside its chart, so the two mechanisms are fully independent: ELO degradation (curve A) reduces the Maia ELO the engine is queried at, while distribution restriction (curve B) keeps the ELO fixed but narrows which slice of Maia's move distribution is available. You can run either alone, both together, or switch both off for constant strength. When a curve is off it is flat at its reference value (curve A at the configured ELO, curve B at 100%), and its slider no longer re-anchors the other curve.

```
Curve interpolation (log-x space):
pts sorted by x. For xSec in [pts[i].x, pts[i+1].x]:
t = (ln(xSec) − ln(pts[i].x)) / (ln(pts[i+1].x) − ln(pts[i].x))
y = pts[i].y + t × (pts[i+1].y − pts[i].y)
```

Curve A — ELO Degradation

Maps think time (s) → effective Maia ELO. At the time control's nominal pace the bot queries its configured ELO; as think time shrinks the ELO drops, producing weaker move distributions.

Seeded on the Regan time-rating model — a power-law fit of IM Kenneth Regan's rating-loss vs time-control data (cse.buffalo.edu/~regan), from his Perpetual Chess Podcast interview with Ben Johnson (youtube.com/watch?v=UsEIBzm5msU). Regan's x-axis (total minutes for a 60-move game) is numerically identical to seconds per move, so the anchor uses EXPECTED_MOVES = 60:

```
D(s) = 339 − 1442 · s^(−0.283) // rating loss, ≈0 at 165 s/move
anchorSec = baseSec / 60 + incrementSec // 15+0 → 15, 5+3 → 8; untimed → flat curve
floor = max(600, E0 − maxDrop) // Max ELO drop slider caps the fall
seed(s) = clamp(E0 + D(s) − D(anchorSec), floor, min(E0, 2600))
```

- Degradation-only: the min(E0, …) cap means banking time never plays above the configured ELO
- Max ELO drop slider (0–600, default 300) caps how far the curve falls — it bottoms out at max(600, E0 – maxDrop) instead of running to Maia's 600 floor, and re-seeds the knots to match
- 8 draggable knots seeded on this curve — one exactly at the anchor (the curve's corner), the rest log-spaced down the dive where the power law bends hardest; drag any point to reshape
- The engine always runs the spline: effectiveELO = evalPressureCurve(curveA, thinkSec) clamped to [600, 2600]; below the 1 s chart edge it holds its 1 s value. Re-seeds on ELO, time-control, or Max-drop changes.

The rough thinkSec estimate (computed before the Maia query) is used for ELO selection. The precise thinkSec (after the engine responds) is used for curve B and temperature.

Hybrid Maia slots carry their own ratings, so they take the curve's RELATIVE drop instead: the spline's relaxed top minus its value at this thinkSec, subtracted from the slot's own ELO (pressureSlotEloByThink). Relaxed think = no drop; under pressure all slots degrade by the same amount, preserving their identity gap.

Curve B — Distribution Cutoff

Maps think time (s) → upper percentile cutoff of the move candidate list. 100% = full distribution (top candidate down to last). As think time falls the cutoff drops — forcing the bot to pick from a narrower top slice of the distribution (higher-probability, safer-looking moves).

```
effectiveUpperPct = pressureEffectiveDayUpperByThink(thinkSec)
= min(botDayUpper, evalPressureCurve(curveB, thinkSec))
```

- X axis: think time per move in seconds (log scale)
- Y axis: upper percentile 0–100%
- Combined with the Move Distribution Range lower bound and Luck shift (§8)

Time Pressure Temperature Boost

In addition to ELO and distribution effects, low think time raises the sampling temperature, making move selection more random under pressure:

```
timePressureTempByThink(baseTemp, thinkSec):
boost = { steady: 0.0, normal: 1.0, panicky: 2.5 }[botTimePressure]
if curveB present:
fraction = 1 − evalPressureCurve(curveB, thinkSec) / 100
else:
fraction = max(0, 1 − thinkSec / 30) # linear fallback
effectiveTemp = baseTemp + fraction × boost
```

botTimePressure modes: "steady" (no boost), "normal" (+1.0 max), "panicky" (+2.5 max).

5. Temperature & Move Sampling

After attractor reweighting, the **conviction pick** decides how the move is chosen: on the **Conviction** share of moves (a slider under the Engine section's distribution chart, 0–100%, default 0%) the bot plays the top-ranked move of the reweighted distribution outright (argmax — its personality's honest first choice); the rest are sampled using temperature-scaled probabilities. Temperature T controls how peaked or flat the distribution is before sampling. Conviction 0% with T = 1.0 and no personality samples Maia's distribution unaltered. Bots saved before the slider existed load at 30%, the fixed share they were built with.

```
pickFromProbs(moveProbs, T):
if rand() < conviction → return argmax of reweighted distribution
else sampleFromProbs(moveProbs, T):
scaled[m] = prob[m] ^ (1/T)
total = Σ scaled[m]
sample r ~ Uniform(0, total)
return first m where cumulative sum ≥ r
```

At T=1.0 the original Maia probabilities are used directly. T < 1 sharpens the distribution (top move more likely). T > 1 flattens it (unlikely moves become relatively more likely). T is clamped to ≥ 0.1 to prevent division issues.

5a. Base Temperature Presets

Preset	T value	Behaviour
Focused	0.4	Strongly favours top 1–2 Maia moves
Neutral	1.0	Samples Maia's full distribution as-is
Varied	1.8	Wider sample — more surprise choices
Wild	3.0	Even low-probability moves get meaningful weight

5b. Hustler Phase Temperature

When the Coffeehouse Hustler personality is active (personalityId = "hustler"), the base temperature is replaced by a phase-sensitive curve:

```
hustlerPhaseTemp():
piecesLeft = count of non-pawn, non-king pieces on board
fraction = clamp(1 – (piecesLeft – 4) / 10, 0, 1)
# 0 at 14 pieces (opening), 1 at ≤4 pieces (endgame)
T = 5.0 + fraction × (0.6 – 5.0)
= 5.0 in opening → 0.6 in endgame
```

14 non-pawn/non-king pieces = full material. At 4 or fewer such pieces, the endgame temperature floor of 0.6 is reached.

5c. Complexity-Adjusted Temperature

For Maia3 mode, the base temperature can optionally be scaled by position complexity (MultiPV evaluation spread):

```
complexityAdjustedTemp(baseT):
if sfCplxScore is null → return baseT
cplxFactor = 0.8 + sfCplxScore × 0.4 # 0.8 at simple, 1.2 at complex
return baseT × cplxFactor
```

sfCplxScore is the normalized std. deviation of the top-N Stockfish evaluations, ranging 0 (all moves equal) to 1 (widely spread evaluations).

6. Strategic Attractors (Centipawmeter)

Strategic attractors reweight each candidate move's probability by multiplying it by exp(logBoost). The total logBoost for a move is the sum of all active attractor contributions. Attractors are zero at centre and scale linearly with slider value (–5 to +5).

6a. CP Budget, Hard Floor and Scale Factor

The CP (centipawn) budget (0–300), shown on the Centipawmeter dial, is a single master gain knob for all attractors. It controls how strongly attractors push the distribution, regardless of the individual slider positions. The budget also names a **real, engine-calculated centipawn ceiling** on how far a move may fall below Maia's most-popular choice — that guarantee is enforced at Stage 8 (§9), not by the internal 150-cp-per-log-unit scale factor below, which only sets the push strength.

The **Hard Floor** (Budget–1000 cp, or Off at the top of the slider) sits directly beneath the Budget and is always ≥ Budget. It is the absolute backstop on every *other* selection mechanism — temperature sampling, the Luck window shift, Bad Day, curve-B time-pressure restriction — enforced by applyHardFloorBackstop (§9c). At Off, nothing is capped: a beginner bot can genuinely hang its queen.

The Engine panel shows both values beside the Elometer with a one-click ELO-scaled recommendation. Budget and Floor have separate curves: Budget ≈ (2700 – ELO)/15 clamped to 10–120 cp (the persistent per-move style spend, a fraction of the rating's typical centipawn loss), Floor ≈ (2900 – ELO)/4.5 clamped to 80–520 cp (the worst single move still in character for the rating).

```
totalAbs = Σ |v| for all attractor and piece sliders
scale = cpBudget / (totalAbs × 150) if cpBudget > 0 and totalAbs > 0
= 0 otherwise

For each move: prob_new = prob × exp(logBoost)
where logBoost = Σ (attractor_v × scale × signal)

At cpBudget=150 with one slider at 1 and all others at 0:
scale = 1/150, signal = 1 → logBoost = 1 → exp(1) ≈ 2.7× boost
```

Allocating 150 cp of budget on a single maximally-set attractor produces roughly a 2.7× probability boost on moves that fully satisfy that attractor.

6b. Attractor Formulas

Attractor	Left (–) / Right (+)	Signal formula	Notes
Chaos Agent / Simplifier	– Simplifier → low-σ moves + Chaos Agent → high-σ moves	logBoost += chaosVal × scale × signal (signal from MultiPV σ)	Requires MultiPV Stockfish probe. σ = std. dev. of top-N eval scores.
Complicate / Simplify winning	– Simplify when winning + Complicate when winning	Active when eval ≥ +100 cp. Boosts moves that increase or decrease MultiPV variance.	Only fires in clearly winning positions. Combined with Chaos Agent for full effect.
Space Cadet / Space Waster	– Space Waster → cede space + Space Cadet → reduce weak sq.	delta = currentWeakSq – simWeakSq logBoost += v × scale × tanh(delta/5)	Full board attack map computed on simulated position. Captures discovered attacks and blocking moves. delta > 0 = fewer weak squares after move.
Fort Knox / Glass Cannon	– Glass Cannon → exposed pieces + Fort Knox → more defenders	delta = simTotalDefs – currentDefs logBoost += v × scale × tanh(delta/3)	Counts sum of defensive coverage across all bot pieces. Full board attack map on simulated position.
Gambito / Gambit Shy	– Gambit Shy → avoid sacrifices + Gambito → follow gambit lines	Opening only (< 20 half-moves). ECO: logBoost += v × scale if move is ECO gambit continuation. Fallback: pawn to attacked, undefended sq.	ECO gambit line match takes priority. Structural fallback if ECO data not yet loaded.

Trade Seeker / Trade Avoider	– Trade Avoider → no captures + Trade Seeker → captures	Capture: logBoost += v × scale × tanh(ctx) Threat: logBoost += v × scale × min(tanh(ctx × newThreats / 2), tanh(ctx))	Non-captures scored by number of new opponent-piece threats created from destination square, capped at the capture score — offering a trade never outscores making one.
Rigid / Loose Pawn Structure	– Loose → open mobile structure + Rigid → tighter structure	Own pawn moves only. delta = currentPenalty – simPenalty logBoost += v × scale × tanh(delta)	Penalty = pawn islands + doubled pawns + isolated pawns. delta > 0 = move tightens structure.
Coffeehouse Hustler / Overthinker	– Overthinker → longer think time + Hustler → shorter think time	Applied to think time (not prob): thinkMs ×= (1 – v × 0.15)	At +5: ×0.25 (4× faster). At –5: ×1.75 (75% slower). Feeds into pressure curves as actual thinkSec.
Luck / Bad Day	– Good day → top of distribution + Bad day → bottom of distribution	Shifts distribution window: lo = botDayLower – v × 4 hi = _pressureUpper – v × 4	See §8 Move Distribution Range for full description.
Panicky / Calm under pressure	– Calm → no temperature boost + Panicky → larger T boost under pressure	Modifies timePressure boost mode. (steady / normal / panicky)	See §4 Pressure Temperature Boost.
Attacker / Peacemaker	– Peacemaker → avoids captures, then threats + Attacker → captures first, then threats	Δ = attacks on opp. pieces after – before (+ attacks on the captured piece) capture: 0.75 + 0.25 × tanh(Δ / 2) quiet: 0.5 × tanh(Δ / 2) logBoost += v × scale × tier	Every capture outranks every quiet move; a capture keeps credit for the attacks it cashed in. Presets wanting calm that still trades use Tension → Defuser.

All attractor signals that use tanh map their input to a smooth –1..+1 range, preventing any single move from receiving an unbounded boost.

7. Piece-Type Attractors

Six independent sliders (–5 to +5) bias the bot toward or away from moving each piece type. The formula is the same for all six:

```
logBoost += pieceVals[pieceType] × scale
```

The piece type is determined from the moving piece before the move. The signal is 1 (or 0 for no boost) — there is no continuous signal function as with most strategic attractors. Scale is shared with the strategic attractors so the CP budget controls piece attractor strength alongside everything else.

Piece	Left (–) behaviour	Right (+) behaviour	Best combined with
Pawn ■	Suppress pawn moves	Prioritise pawn advances	Rigid structure (connected chains)
Knight ■	De-prioritise knight hops	Knight manoeuvres first	Rigid structure (closed positions)
Bishop ■	Keep bishops passive	Diagonal play, bishop pair	Loose structure (open diagonals)
Rook ■	Passive rooks, avoid early trades	File seizure, rook lifts	Open files, trade-seeker
Queen ■	Conservative queen, avoid exposure	Active queen, queen-led attacks	Chaos Agent (keeps complications)
King ■	King safety, stay castled	King activity, endgame aggression	Endgame positions, low piece count

8. Move Distribution Filter & Luck

The Move Distribution Range dual slider selects which segment of the Maia probability-ranked candidate list the bot samples from. Candidates outside the [lower, upper] percentile window are excluded before temperature sampling and attractor reweighting.

> *The ranking is popularity, not engine quality — Maia probability is how often players at the selected rating choose a move. A strong move few players see can sit low in the distribution; real move-quality enforcement is the engine check in §9.*

8a. Percentile Band Filtering

```
sorted candidates by probability (highest first)
cumulative probability sum = total

band.lo = (lo/100) × total
band.hi = (hi/100) × total

Include move m if: cumulative_end(m) > band.lo AND cumulative_start(m) < band.hi
```

Default: lo = 0%, hi = 100% (full distribution). Setting lo = 20%, hi = 60% samples only the middle tier — avoiding the top moves (too accurate) and the worst moves (random blunders).

8b. Luck Attractor Shift

The Luck attractor shifts both bounds of the quality window simultaneously:

```
lo_effective = botDayLower − luckVal × 4
hi_effective = pressureUpper − luckVal × 4

luckVal > 0 (Bad day): window shifts down → samples lower-ranked moves → worse play
luckVal < 0 (Good day): window shifts up → samples top-ranked moves → sharper play
```

pressureUpper is the curve-B degraded upper bound, so luck shift compounds with time pressure effects.

8c. Min-Probability Filter

A single absolute-popularity filter prunes the candidate list before the distribution range is applied. (The former relative "blunder-limit" cutoff — which pretended a probability ratio was a centipawn value — has been removed; centipawn enforcement is now the engine-calculated check in §9.)

```
absFloor = minProbPct / 100
keep only moves with prob ≥ absFloor
(if that empties the list, keep the single most-popular move)
```

- minProbPct (the "Exclude lowest %" slider): absolute probability floor; removes near-zero candidates. This is an honest distribution control, expressed as a percentage, with no pretence of measuring quality.

9. CP-Budget Acceptance, Degradation Guard & Hard Floor (Engine-Calculated)

Maia probability is popularity, not quality. Three post-sampling stages use a shallow Stockfish probe to hold the played move to a real centipawn standard. The first two run only for personality bots (they need the Stage-5 reweighting flag); the Hard Floor backstop (§9c) covers picks from *any* mechanism. All run only when the pick differs from Maia's most-popular move, and all are fail-open — any probe failure or timeout leaves the pick unchanged. The probe reuses the MultiPV complexity machinery with a searchmoves restriction (depth ~10; the timeout scales with candidate count, capped at 4.5 s).

9a. CP-Budget Acceptance — applyCpBudgetAcceptance()

Turns the Centipawmeter into a real centipawn ceiling. When personality reweighting produced a pick different from the most-popular move, the pick, the popular move, and a wide slice of the personality's preference order (up to 15 moves, one combined MultiPV probe) are evaluated together. The pick is accepted only if it loses no more than the CP budget versus the popular move; otherwise the bot walks down its own preference order until one fits, falling back to the popular move itself (0 cp by definition).

```
topMove = argmax Maia probability
if pick == topMove or reweighting not applied → keep pick
evals = SF probe over [topMove, pick, next favourites]
for m in [pick, ...favourites]:
if evals[topMove] − evals[m] ≤ cpBudget → play m
else → play topMove
```

Stalemate-seeking picks (§10) are deliberately exempt — throwing material is the whole point, so the budget must not veto them.

9b. Degradation Guard — applyDegradationEvalGuard()

The builder switch **Never play better than the most popular move** (Personality → CP Budget, off by default). When on, a sampled pick that evaluates better than Maia's top move is swapped back to the top move, so probability-steered picks (temperature, Bad Day) can never out-perform the mainstream move. Personality picks may still beat it by up to the CP Budget. When off, the bot can play the strong-but-rare move Maia itself sometimes plays. Bots saved before the switch existed load with it on, as they played.

```
if switch off or pick == topMove → keep
evals = SF probe over [pick, topMove]
allowance = Budget if the personality reshaped this move, else 0
if eval(pick) − eval(topMove) > allowance → play topMove
```

Grandmaster Bad Day picks the lowest-probability move above the min-probability slider's floor (the preset sets 4%), and its preset turns this switch on: if that move happens to be strong (a shot few players see), the guard swaps it back to the mainstream move — the bot is never accidentally brilliant.

9c. Hard Floor Backstop — applyHardFloorBackstop()

Runs last in the pick flow and bounds how far below Maia's most-popular move ANY final pick may fall — whatever produced it: temperature sampling, the Luck window shift, Bad Day, curve-B time-pressure restriction, or plain sampling variance. Skipped when the pick was already engine-calculated within the Budget (Budget ≤ Floor by invariant), when the Floor slider is at Off (≥ 1000 cp), and for stalemate-seeking desperation picks. Reuses the degradation guard's probe when one was taken for the same position; otherwise pays for one shallow probe of its own.

```
floor = hardFloorCp (panel slider, Budget–1000; ≥ 1000 → Off)
if budget-verified this move, or floor Off, or pick == topMove → keep
evals = SF probe over [pick, topMove] (or reuse guard probe)
if evals[topMove] − evals[pick] > floor → play topMove
else → keep pick
```

10. Draw & Desperation Behaviours

Four opt-in behaviours (in the Move Timing section's "Draws & desperation" group) let a bot handle draw offers, offer draws, and swindle when lost — all judged the way a human of the bot's strength would, not at raw engine accuracy.

10a. Perceived Evaluation — botPerceivedAdvantageCp()

Every draw decision uses the bot's **perceived** advantage, not Stockfish's. A shallow eval probe gives the true centipawn advantage from the bot's side; that value is then blurred by rating-scaled Gaussian noise plus a mild self-optimism bias. A novice genuinely misjudges; a master barely does.

```
sigma(elo) = 15 + 700 × exp(−elo / 500)
≈ 225 cp at 600 ELO, ≈ 50 cp at 1500, ≈ 19 cp at 2600
perceived = trueAdv + gauss()×sigma + 0.25×sigma
```

So a 700-rated bot can believe a lost position is fine (and decline your draw, or offer one from a losing position it thinks is level); a 2400 is almost never fooled.

10b. Accepting & Offering Draws

Behaviour	Control	Rule
Accepts draw offers	toggle + accept-margin (0–300 cp)	On your ½ offer the bot accepts iff its *perceived* advantage ≤ margin. Declines are wordless (a human just plays on — no eval is announced), and a bot that knows it is worse may still decline, hoping for a blunder.
Offers draws	toggle + level-band (±cp) + from-move	After its move, in a position that *feels* level (|perceived| ≤ band) and past the move threshold, it occasionally offers — never twice within 12 plies.

10c. Clock Awareness

Draw decisions read the clock (only when the increment is under 10 s, since nobody flags with a healthy increment):

- **Opponent about to flag** (under 20 s, bot comfortably ahead on time): the bot declines instantly with no eval probe and never offers — it is playing for the win on time.
- **Bot itself about to flag** while the opponent is comfortable: it becomes far more agreeable — accept margin widens by 200 cp and its offer band by 150 cp, grabbing the half point even from better positions.

Untimed games and games with a ≥ 10 s increment ignore the clock entirely.

10d. Seek Stalemate When Losing — _maybeStaleSeek()

A desperation swindle mode. Once the engine eval says the bot is worse than the threshold and the game has passed the configured move number, candidate weights are re-shaped toward the classic stalemate-trap recipe:

- Moves that reduce the bot's own future mobility (fewer legal replies afterwards → closer to a stalemate shape) are boosted.
- Desperado moves — offering the moved piece for capture — are boosted in proportion to the piece value, most of all when the piece is undefended (dumping the queen is the point).

```
active if staleSeek AND fullmove ≥ fromMove AND eval ≤ −thresholdCp
weight ×= exp( mobilityBoost + desperadoBoost )
```

These picks are exempt from the §9 CP-budget check. Arming this behaviour also forces the complexity/eval probe every move so the trigger stays current.

11. Preset Personalities

Each preset personality is a named collection of attractor values; loading one sets all sliders to those values (–5 to +5 scale). Some presets also reconfigure the engine, opening repertoire, or draw behaviour and confirm first via a dialog. The table below shows each personality's notable settings.

Personality	Tag	Notable attractor values	Character
Captain Entropy ■	Chaos	chaos:+4, compwin:+4, gambito:+3 trade:+2, spacecadet:+2 hustle:–3, structure:–3	Seeks complications, gambit lines, and tactical chaos. Thinks longer per move (hustle –3).
Norm ■	Order	structure:+4, fortkx:+3, spacecadet:0 chaos:–4, compwin:–4 gambito:–2, hustle:+2, tension:–2	Closes positions, defends solidly. Simplifies when winning. Slightly faster pace.
Attacky McTackerson ■	Captures	attacker:+5, trade:+3, hustle:–2 (nothing else shares the budget)	Takes whatever it can within the cp ceiling; with nothing to take, piles attacks onto your pieces. Slightly deliberate (hustle –2).
Overthinker ■	Methodical	structure:+3, spacecadet:+3, fortkx:+2 hustle:–4, pressure:–3 chaos:–3, compwin:–2	Maximises board control and structure. Thinks long on every move (hustle –4). Likely to accumulate time pressure in classical games.
Coffeehouse Hustler ■	Fast	hustle:+5, pressure:+2, luck:+2 gambito:+2, hustle:+5 structure:–2	Plays fast and loose. Phase temperature: T=5 opening → T=0.6 endgame. At hustle +5: think time ×0.25 of baseline.
The Blunderer ■	Human	luck:+3, pressure:+5	Plays normally but cracks unpredictably under pressure. Luck shifts toward lower-ranked moves.
The Hoarder ■	Material	gambito:–3, trade:–3, fortkx:+2 chaos:–2, hustle:+2, luck:–2 tension:–2	Never sacrifices material. Avoids trades and gambits. Slight faster pace.
Pawn Chain Gang ■	Pawns	structure:+5, fortkx:+1, hustle:+1 chaos:–1, compwin:–2, gambito:–1	Maximises pawn structure. Slightly faster pace.
Spite Check †	Checks	chaos:+3, compwin:+2, attacker:+3 trade:+2, hustle:–2	Always prefers checking moves. Builds up attacks. Slower deliberate pace (hustle –2).
Clock Watcher ■	Clock	compwin:–2, trade:–1, pressure:–3	Conservative when ahead on time, reckless when behind. Calm under pressure (pressure –3).
Grandmaster Bad Day ■	Human	luck:+5, pressure:+2 (Maia 3 @ 2400, picks lowest prob move above 4%)	Strong player having an off day. The preset switches on the §9 degradation guard, which prevents it from ever accidentally choosing a strong move — never brilliant by mistake.
Drunken Master ■	Human	attacker:+4, chaos:+3, gambito:+3 (Hybrid: 50% Maia 2400 + 50% Maia 1000 per move)	Each move rolls one of two Maia strengths — brilliant one move, blundering the next. Plain hybrid config, so the two slots are freely re-tunable.
The Drawmeister ½	Human	trade:+4, fortkx:+3, structure:+2 chaos:–4, gambito:–3, tension:–3 (accepts + offers draws)	Plays for the half point: simplifies, fortresses up, offers/accepts draws (§10), and installs solid drawish repertoires for both colours (London / Exchange Slav / Four Knights; Petroff / Berlin / Slav).

Hustle attractor sign convention: +5 = Coffeehouse Hustler (fast, thinkMs ×0.25). –5 = Overthinker (slow, thinkMs ×1.75).

12. Time Controls

Time controls are defined in TIME_CONTROLS and selected from the bot panel grid. The grid is divided into Blitz (≤5 min), Rapid (10–30 min), Classical/Correspondence (60 min+), Infinite (untimed), and a special 90+30 column for FIDE/Candidates format.

Format	Base time	Increment	Bonus	Bonus at	Inc from
Bullet 1+0	1 min	0 s	—	—	—
Blitz 3+2	3 min	2 s	—	—	—
Blitz 5+0	5 min	0 s	—	—	—
Rapid 10+0	10 min	0 s	—	—	—
Rapid 15+10	15 min	10 s	—	—	—
Classical 30+0	30 min	0 s	—	—	—
Classical 60+0	60 min	0 s	—	—	—
Classical 90+0	90 min	0 s	—	—	—
FIDE 90+30 *	90 min	30 s	+30 min	Move 40	Move 41
Untimed	∞	0 s	—	—	—

** 90+30 (FIDE / Candidates): 90 minutes for first 40 moves, then +30 minutes added to both clocks at move 40. 30-second increment applies from move 41 onward. The clockBonusApplied flag prevents the bonus from being awarded twice.*

Bonus Time Implementation

```
clockAfterMove():
fullMoveNum = ceil(gameMovesAlgebraic.length / 2)
if tc.bonusSecs AND fullMoveNum ≥ tc.bonusAtMove AND NOT clockBonusApplied:
clockTimeW += bonusSecs (capped at 59940 s)
clockTimeB += bonusSecs
clockBonusApplied = true
if clockInc > 0 AND fullMoveNum ≥ tc.incFromMove:
add increment to side that just moved
```

13. Human Behaviour Flags

Human behaviour flags add realistic timing irregularities and clock-pressure responses. Each flag is independently toggleable. They modify think time or move selection in specific situations.

Flag	When active	Effect
Move Blink (botBehavBlink)	Maia3 mode only. Position entropy < 0.5 (forced or obvious move)	Returns 200–500 ms random pause immediately, skipping full think time calculation. Simulates a human instantly playing the obvious recapture or forced reply.
Reconsider (botBehavReconsider)	15% random chance per move	Multiplies computed think time by 1.5–2.5×. Simulates the human hesitation of starting to play a move then reconsidering.
Clock Mirror (botBehavClockMirror)	Opponent clock < bot clock × 0.6	Halves think time when the opponent is significantly lower on time. Simulates a human speeding up to maintain clock advantage.
Clock Weaponizer (botWeaponizerEnabled)	Bot clock ahead of opp by more than leadMs	Returns 0 ms think time AND drops Stockfish to floor level. Maximises time pressure on the opponent by playing instantly when already ahead on clock. leadMs configured by Weaponizer Lead Time slider.
Can Flag (botCanFlag)	Always on or off	When OFF: think time capped at max(200 ms, clockMs – 3000 ms). Prevents the bot from flagging itself. When ON: no such safeguard.

14. Appendix — Play From Any Position

Beyond bot configuration, games are no longer locked to the standard start. Any finished game, or any loaded PGN, can be reviewed move-by-move and resumed as a new game — against a bot or a friend — from any position along the way.

14a. Review & Replay

- After any game a **Review** button enters replay of the moves just played; step to any position. The last move is highlighted exactly as in live play.
- On the Expert board, notation moves are **clickable** whenever no game is live — clicking one jumps the board to that position.
- Loaded PGNs support a [FEN] set-up header; a position-only PGN opens directly at that position, ready to play.

14b. Resume Play From Here

- **Play from here** starts a bot game from the shown position (the Bot Builder opens; the configured bot then plays on from that point). Repertoire book moves are skipped, but the Lichess explorer stays active — it queries by position, so a classic position still draws real human move frequencies.
- **Invite from here** stages the position in the 2-player panel (a banner shows what is staged) so the initiator can first pick their colour — White, Black, or Random, regardless of whose move it is — then Start Private Game creates the room (invite link/code only; never posted to the open-challenge board). The server assigns colours and relays a sanitised start position so both boards agree; rematches restart from the same position with colours swapped.
- The "Play as" choice applies to every game you create, and the same freedom exists for bot continuations via the Bot Builder's colour picker — replay a position several times from either side.
- Any control that would end a game in progress (starting a new bot or online game, loading a PGN) asks for confirmation first, and every game start fully clears the previous game's replay state.
- Saving a from-a-position game writes SetUp/FEN tags (or a complete move list when the prefix is known), so a game can be saved and continued days — or years — later.

End of brief. For implementation details see src/50-bot-engine.js (engine), src/60-bot-ui.js (config application), src/10-app-shell.js (replay / draws / multiplayer), server.js (private rooms), and bot-control-panel.html (UI and presets).

Generated July 2026 — Blundermind Bot Controls, dev branch.